



Data Build Tool

The T in ELT — write SQL,
build tested, documented
data pipelines automatically.

What Problem Does dbt Solve?

Before dbt

- SQL scripts scattered in emails / Confluence
- No version control — who changed what?
- No automated testing — silent bad data
- No lineage — unclear dependencies
- Hard to onboard new team members

With dbt

- SQL models in Git — versioned, reviewable
- DAG built automatically from ref() calls
- Built-in data quality tests (schema.yml)
- Auto-generated docs + lineage graph
- Runs inside your warehouse — no extra infra

dbt Project Structure

```
my_dbt_project/
```

```
├─ models/
```

```
│   └─ staging/
```

```
│   └─ marts/
```

```
├─ seeds/
```

```
├─ macros/
```

```
├─ tests/
```

```
├─ snapshots/
```

```
├─ dbt_project.yml
```

```
└─ profiles.yml
```

models/

SQL files = your transformations. Each .sql file becomes a table or view in Snowflake.

seeds/

CSV files loaded as static lookup tables (e.g. country codes, mappings).

macros/

Reusable Jinja functions — like helper functions you call from multiple models.

tests/

Custom SQL tests. Built-in tests (not_null, unique) live in schema.yml.

dbt_project.yml

Project config — name, version, default materialization per folder.

Models & Materializations

How dbt persists each model in the warehouse

view

Default

Creates a SQL view on every dbt run. No data stored — computed at query time. Best for staging models you always want fresh.

table

Full Refresh

Drops and recreates a physical table on every run. Fast to query but slow to build. Best for marts queried often by analysts.

incremental

Efficient

Only processes new/changed rows using `is_incremental()`. Avoids full rebuilds. Best for large event or log tables.

ephemeral

CTE Only

Not materialized at all — inlined as a CTE during the run. Exists only in memory. Best for intermediate steps never queried directly.

ref() and source() — The DAG

Raw Source
(Snowflake table)

`source()` →

stg_orders
(staging model)

`ref()` →

fct_orders
(mart)

`source()` — stg_orders.sql

```
-- Reference a RAW table (not a dbt model)
SELECT
    order_id,
    customer_id,
    amount,
    created_at
FROM {{ source('raw', 'orders') }}
```

-- 'raw' = source name in sources.yml
-- 'orders' = table name

`ref()` — fct_orders.sql

```
-- Reference ANOTHER dbt model
SELECT
    o.order_id,
    c.name,
    o.amount
FROM {{ ref('stg_orders') }} o
JOIN {{ ref('stg_customers') }} c
    ON o.customer_id = c.customer_id
-- dbt builds the DAG automatically
```

Testing in dbt

not_null

No NULLs allowed

unique

All values distinct

accepted_values

Value in defined list

relationships

FK exists in ref table

schema.yml

```
version: 2
models:
  - name: stg_orders
    columns:
      - name: order_id
        tests:
          - not_null
          - unique
      - name: status
        tests:
          - accepted_values:
              values:
                - 'placed'
                - 'shipped'
                - 'delivered'
```

Run with: dbt test

Runs all schema tests across all models. Fails loudly if data quality breaks.

Where tests live

Built-in tests → schema.yml. Custom SQL tests → tests/ folder (SQL that should return 0 rows).

Why it matters

Catches bad data before analysts see it. Data trust > data speed.

Essential dbt Commands

dbt run

Executes all models — builds views/tables in the warehouse

dbt docs generate

Builds data catalog with lineage DAG and column descriptions

dbt test

Runs all schema and custom data quality tests

dbt docs serve

Opens the catalog site locally — see the full DAG visually

dbt seed

Loads CSVs from seeds/ into the warehouse as tables

dbt run --select stg_orders

Run a single model only — critical for development speed

dbt in One Sentence

"Write SQL. dbt handles the rest."

`models/`

`ref() & source()`

`dbt test`

`dbt run`

Questions?